

In this issue

© Springer Science + Business Media, Inc. 2006

The first paper, titled “An Experimental Evaluation of a Higher-Ordered-Typed-Functional Specification-Based Test-Generation Technique,” by A. Sinha and C. Smids, investigates the usability and performance of a specific model-based test technique (HOTT). The main result from the experiment is that HOTT-based test suites provides enhanced performance without compromising usability when compared to traditional EFSM-based test suites. Beyond its specific experimental results, this paper is a very good example of a methodology for demonstrating how to assess testing techniques by investigating how human factors may affect their usability.

The second paper, titled “Integrating Agile Software Development into Stage-Gate Managed Product Development,” by D. Karlström and P. Runeson, uses a fully traceable, qualitative, case study methodology, to investigate the use of eXtreme Programming (XP) within the context of a stage-gate project management model. The case studies involve the development of two large independent embedded software systems. The study results show it is possible to reconcile the two approaches in an effective manner. This paper offers a good example of how to run a qualitative study to yield reliable, useful results, which is important methodologically since many types of software engineering problems can only be investigated in this way.

The third paper, titled “Using Patterns for the Refinement and Translation of UML Models: A Controlled Experiment,” by C. Bunse, reports on a controlled experiment that investigates how SORT, a method for the systematic refinement and translation of UML Diagrams into code, affects effort and the quality attributes of understandability and verifiability. This study is a good example of a within-subject experimental design that could be easily reused for validating other design techniques.

The fourth paper, titled “Investigating The Extreme Programming System—An Empirical Study,” by P. Sfetos, L. Angelis, and I. Stamelos, reports on a large scale field study, involving 15 companies and 30 software developers and managers. The goal of this study was to identify the difficulties faced when adopting XP practices and what were the main success drivers. From a general standpoint, such studies are crucial in software engineering as they allow researchers to better understand the problems from a practical standpoint and allow practitioners to determine what issues and benefits they can expect when adopting a technology, and under what conditions.

The fifth paper, titled “An Experiment on Linguistic Tool Support for Consolidation of Requirements from Multiple Sources in Market-Driven Product

Development,” by J. Natt och Dag, T. Thelin, and B. Regnell, aims to evaluate a tool to support (based on linguistic engineering techniques) the consolidation of requirements. When new requirements are elicited, they must be consolidated with the old requirements. A laboratory setting experiment is reported that builds on previously reported industrial case studies and shows that such a tool is indeed useful to improve correctness and efficiency when consolidating two sets of requirements. This is another very good example of a carefully designed, controlled experiment to assess the cost-effectiveness of a tool.

It is interesting to note that the five papers presented above cover a large spectrum of the software life cycle, from requirements engineering to testing. They also make use of a variety of empirical methods, ranging from quantitative controlled experiments to qualitative analysis in an industrial setting. This issue of the journal illustrates particularly well the level of maturity achieved by the field of empirical software engineering. The studies reported address important practical issues, provide solid results, and make use of a large variety of state-of-the-art empirical methods in a very rigorous manner.

We hope you will learn as much from these papers as we did.

Lionel Briand
Victor Basili